DEPARTEMEN AKUNTANSI
FAKULTAS EKONOMIKA DAN BISNIS UNIVERSITAS DIPONEGORO

Jawaban Ujian Tengah Semester Pemrograman / Coding Akuntansi

General Ledger Double-Entry, Trigger Persediaan Perpetual, dan Purwarupa Voucher Kas Keluar dengan Audit Trail

Nama	**Bintang Rifqi Maulana DS**
NIM	**12030123140235**
Program Studi	S1 Akuntansi
Dosen Pengampu	Totok Dewayanto
Semester	Gasal 2026/2027
Tanggal	13 Oktober 2026

Repositori GitHub: https://github.com/bintangrifqi1-bit/uts-ledger-kas-bintang
Demo web (GitHub Pages): https://bintangrifqi1-bit.github.io/uts-ledger-kas-bintang/
Basis data: Supabase (PostgreSQL), project `uts-ledger-kas`

Sifat ujian: Open Book / Open AI / Open Code Sandbox — Alat bantu: Gemini Canvas, NotebookLM, PostgreSQL (Supabase)

Ringkasan Pengerjaan

Seluruh jawaban diimplementasikan dan **dijalankan sungguhan** di Supabase (PostgreSQL), bukan hanya ditulis. Satu basis data yang sama dipakai oleh tiga soal. **Soal 1** membangun buku besar dengan pengecekan debit = kredit di dalam database. **Soal 2** menambahkan trigger persediaan yang otomatis menjurnal Persediaan/HPP ke buku besar itu. **Soal 3** membangun form web Voucher Kas Keluar yang menulis ke buku besar lewat fungsi RPC, dilengkapi kunci POSTED, storno, dan hash berantai.

Artefak	Lokasi di repositori
DDL & constraint (Soal 1)	`sql/01_schema_gl.sql`, ERD: `docs/erd.md`
Trigger persediaan (Soal 2.1)	`sql/02_inventory_trigger.sql`
Concurrency (Soal 2.2)	`sql/03_concurrency.sql`
Audit trail & RPC (Soal 3.2)	`sql/04_audit_trail.sql`
Purwarupa web (Soal 3.1a)	`index.html` (di-host GitHub Pages)
Log prompt AI (Soal 3.1)	`ai-logs/01-gemini-canvas.md`, `ai-logs/02-notebooklm-verifikasi-psak.md`
Bukti pengujian	`sql/06_tests.sql`, `docs/hasil-uji-sql.md`, `docs/hasil-uji-api.json`

Arsitektur

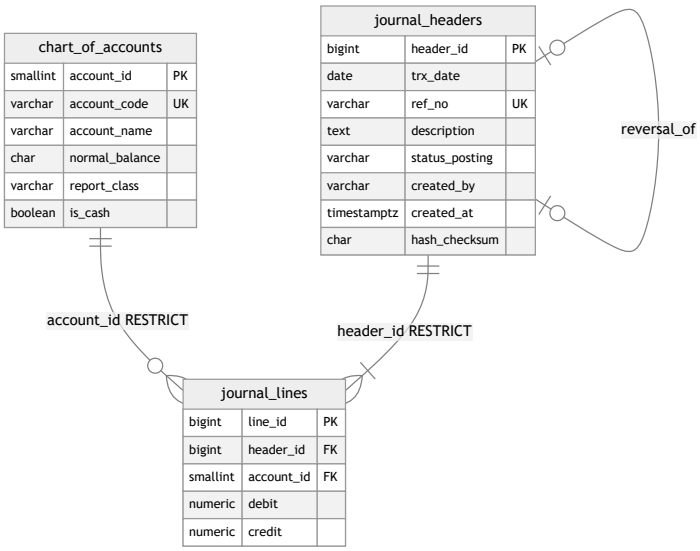


Hasil pengujian (ringkas)

- 16/16 PASS uji otomatis `run_ledger_tests()` (Bagian 5).
- Insert/update/delete langsung lewat REST API dengan anon key: **ditolak** (*permission denied*). Voucher tidak seimbang lewat RPC: **ditolak**.
- Dua kasir menjual BRG-05 (stok 1) secara paralel: satu berhasil, satu **ditolak**. Stok akhir 0, tidak minus.
- Storno BKK-202610-0005 berhasil, storno ganda ditolak, dan `verify_ledger()` menyatakan 13 jurnal dengan rantai hash valid.

Soal 1 — Desain Arsitektur Data, Normalisasi & Asas Berpasangan

1.1 Skema relasional (ERD)



Relasinya: **satu header memiliki banyak line** (1:N, minimal 2 baris). **Satu akun dipakai oleh banyak line** (1:N). Header storno menunjuk ke header asal lewat `reversal_of`. Kolom audit (created_by, hash_checksum, dll.) ditambahkan di Soal 3.2. ERD lengkap ada di `docs/erd.md`.

1.2 DDL PostgreSQL dan kekangan integritas

```
create table chart_of_accounts (
    account_id      smallint generated always as identity primary key,
    account_code    varchar(10)  not null unique,
    account_name    varchar(100) not null,
    normal_balance  char(1)      not null check (normal_balance in ('D', 'K')),
    report_class    varchar(10)  not null check (report_class in ('NERACA', 'LABA_RUGI')),
    is_cash         boolean      not null default false);

create table journal_headers (
    header_id       bigint generated always as identity primary key,
    trx_date        date         not null,
    ref_no          varchar(30)  not null unique,
    description     text         not null,
    status_posting  varchar(10)  not null default 'DRAFT' check (status_posting in ('DRAFT', 'POSTED')));

create table journal_lines (
    line_id     bigint generated always as identity primary key,
    header_id   bigint   not null references journal_headers (header_id)    on delete restrict,
    account_id  smallint not null references chart_of_accounts (account_id) on delete restrict,
    debit       numeric(18, 2) not null default 0,
    credit      numeric(18, 2) not null default 0,
    constraint ck_debit_non_negative  check (debit  >= 0),
    constraint ck_credit_non_negative check (credit >= 0),
    constraint ck_not_both_zero       check (debit > 0 or credit > 0),   -- nol-nol ditolak
    constraint ck_single_side         check (debit = 0 or credit = 0));  -- satu baris satu sisi
```

Menjawab akar masalah kasus. CHECK hanya bisa memeriksa satu baris, padahal keseimbangan adalah sifat satu *jurnal* (gabungan beberapa baris). Karena itu ditambahkan **constraint trigger yang ditunda sampai COMMIT**. Transaksi yang dikirim langsung lewat API pun tetap di-rollback bila Σdebit ≠ Σkredit atau barisnya kurang dari dua.

```
create constraint trigger ctrg_lines_balanced
    after insert or update or delete on journal_lines
    deferrable initially deferred
    for each row execute function fn_assert_journal_balanced();
-- fungsi: select sum(debit), sum(credit), count(*) ... where header_id = v_id;
--         if v_n < 2 or v_dr <> v_cr then raise exception 'Jurnal #% ditolak: tidak seimbang ...'
```

1.3 Analisis audit: Header-Line vs tabel jurnal tunggal

Aspek	Model Header-Line (dipakai)	Flat journal table
Normalisasi 3NF	Atribut dokumen (tanggal, no. bukti, uraian, status, pembuat) hanya bergantung pada `header_id` dan disimpan **sekali**. Line hanya berisi atribut milik baris. Tidak ada ketergantungan transitif. Nama akun dan saldo normal tersimpan di COA.	Tanggal, no. bukti, uraian, dan nama akun diulang di setiap baris. Ini redundansi yang menimbulkan anomali update (no. bukti bisa berbeda antarbaris) dan anomali delete (menghapus satu baris memutus pasangan jurnal).
Kesatuan transaksi	Satu header = satu dokumen sumber. Keseimbangan diuji per `header_id`, dan status POSTED/kunci berlaku untuk seluruh baris sekaligus.	Tidak ada identitas "jurnal". Pengelompokan baris hanya bergantung pada kesamaan teks no. bukti, sehingga keseimbangan sulit dijamin.
Jejak audit	`created_by`, `created_at`, `posted_by`, `hash_checksum`, dan `reversal_of` melekat sekali di header. Auditor dapat menelusuri dokumen → baris → akun (*vouching*) dan sebaliknya (*tracing*).	Metadata audit harus diulang per baris dan bisa berbeda-beda. Storno sulit dikaitkan dengan jurnal asalnya.
Integritas referensial	FK `ON DELETE RESTRICT`: akun yang sudah dipakai dan header yang memiliki baris tidak bisa dihapus (uji T04).	Akun sering disimpan sebagai teks, sehingga salah ketik membuat akun "baru" tanpa terdeteksi.

Soal 2 — Otomasi Bisnis, Trigger Persediaan & Concurrency

Tabel `inventory_items` dan `inventory_transactions` dibuat persis sesuai naskah. Sebagai lapisan kedua ditambahkan `CHECK (stock_qty >= 0)`. Konvensi qty: PURCHASE dan SALE selalu positif, ADJUSTMENT bertanda (+ selisih lebih, − susut/rusak).

2.1 Trigger `trg_update_stock_and_journal()`

```
create or replace function trg_update_stock_and_journal() returns trigger language plpgsql as $$
declare v_item inventory_items%rowtype; v_delta int; v_amount numeric(18,2);
        v_dr smallint; v_cr smallint; v_hdr bigint;
begin
    if new.trx_type in ('PURCHASE','SALE') and new.qty <= 0 then raise exception 'Qty % harus > 0',
new.trx_type;
    elsif new.trx_type = 'ADJUSTMENT' and new.qty = 0 then raise exception 'Qty ADJUSTMENT tidak boleh 0';
end if;

    -- 2.2: kunci baris item; transaksi lain untuk item yang sama MENUNGGU di sini
    select * into v_item from inventory_items where item_id = new.item_id for update;
    if not found then raise exception 'Item % tidak terdaftar', new.item_id; end if;

    v_delta := case when new.trx_type = 'SALE' then -new.qty else new.qty end;
    if v_item.stock_qty + v_delta < 0 then                                  -- (b) anti-negative inventory
        raise exception 'Stok % (%) tidak cukup: tersedia %, keluar %',
              v_item.item_id, v_item.item_name, v_item.stock_qty, -v_delta;
    end if;
    update inventory_items set stock_qty = stock_qty + v_delta where item_id = new.item_id;   -- (a)

    -- jurnal perpetual otomatis: nilai = |mutasi| x unit_cost
    v_amount := abs(v_delta) * v_item.unit_cost;
    if    new.trx_type = 'PURCHASE' then v_dr := acc_id('1301'); v_cr := acc_id('2101'); -- Dr Persediaan /
Cr Utang
    elsif new.trx_type = 'SALE'     then v_dr := acc_id('5101'); v_cr := acc_id('1301'); -- Dr HPP / Cr
Persediaan
    elsif v_delta > 0               then v_dr := acc_id('1301'); v_cr := acc_id('5102'); -- selisih lebih
    else                                 v_dr := acc_id('5102'); v_cr := acc_id('1301'); -- susut/rusak
    end if;
    insert into journal_headers (trx_date, ref_no, description, source, created_by)
    values (new.trx_date::date, 'INV-' || lpad(new.trx_id::text, 6, '0'), format(...), 'INVENTORY',
'sistem:trigger-persediaan')
    returning header_id into v_hdr;
    insert into journal_lines (header_id, account_id, debit, credit, memo)
    values (v_hdr, v_dr, v_amount, 0, new.item_id), (v_hdr, v_cr, 0, v_amount, new.item_id);
    update journal_headers set status_posting = 'POSTED' where header_id = v_hdr;
    return new;
end $$;

create trigger trg_inventory_after_insert after insert on inventory_transactions
    for each row execute function trg_update_stock_and_journal();
```

Mutasi stok, saldo Persediaan, dan HPP di buku besar diperbarui **dalam satu transaksi database yang sama** dengan baris `inventory_transactions`. Jika salah satu gagal (stok kurang, akun tidak ada, jurnal tidak seimbang), semuanya di-rollback. Dengan begitu tidak ada lagi jeda pemutakhiran yang memicu selisih saat stock opname. Contoh hasil seed: `SALE 12 x Palet Kayu @ 185.000` menghasilkan jurnal INV-000002 Dr HPP 2.220.000 / Cr Persediaan 2.220.000.

> **Batasan:** sisi pendapatan penjualan (Dr Kas/Piutang, Cr Penjualan) tidak dijurnal karena tabel soal tidak memiliki kolom harga jual. Biaya per unit memakai `unit_cost` tetap sesuai struktur soal.

2.2 Penanganan race condition (dua kasir, stok tinggal 1)

Pada isolasi default *READ COMMITTED* tanpa penguncian, kedua sesi dapat membaca stok = 1, keduanya lolos pengecekan, lalu stok menjadi −1 (*overselling/phantom stock*). Teknik yang dipakai adalah **pessimistic row lock `SELECT … FOR UPDATE` di dalam trigger**. Sesi kedua menunggu di baris itu sampai sesi pertama COMMIT, lalu PostgreSQL membaca ulang *versi baris terbaru* (stok 0) sehingga sesi kedua ditolak.

```
-- Kasir A                                    -- Kasir B
BEGIN;                                        BEGIN;
INSERT ... ('BRG-05','SALE',1);
  └ trigger: SELECT ... FOR UPDATE (lock)     INSERT ... ('BRG-05','SALE',1);
  └ stok 1 → 0                                  └ trigger: SELECT ... FOR UPDATE  ⇒ MENUNGGU
COMMIT;  -- lock dilepas                        └ baca versi terbaru: stok = 0
                                                └ RAISE EXCEPTION 'Stok ... tidak cukup' → ROLLBACK

-- Alternatif B: SERIALIZABLE + retry di aplikasi
BEGIN ISOLATION LEVEL SERIALIZABLE;
UPDATE inventory_items SET stock_qty = stock_qty - 1 WHERE item_id = 'BRG-05';
COMMIT;            -- salah satu sesi gagal SQLSTATE 40001, aplikasi wajib mengulang
-- Alternatif C: update atomik bersyarat
UPDATE inventory_items SET stock_qty = stock_qty - 1 WHERE item_id = 'BRG-05' AND stock_qty >= 1 RETURNING
stock_qty;
```

Teknik A dipilih karena berada di trigger, sehingga berlaku untuk **semua jalur input** (aplikasi kasir, API, SQL manual) tanpa logika retry di klien. **Bukti uji nyata:** dua request RPC `record_inventory_trx('BRG-05','SALE',1, hold 2 detik)` dikirim paralel dari browser. Satu selesai (stok_after = 0, 2.010 ms). Yang lain menunggu lock lalu ditolak dengan pesan "*Stok BRG-05 (Timbangan Lantai Digital) tidak cukup: tersedia 0, keluar 1*". Total waktu kedua request 2.065 ms membuktikan bahwa yang kedua memang menunggu yang pertama. Stok akhir = 0.

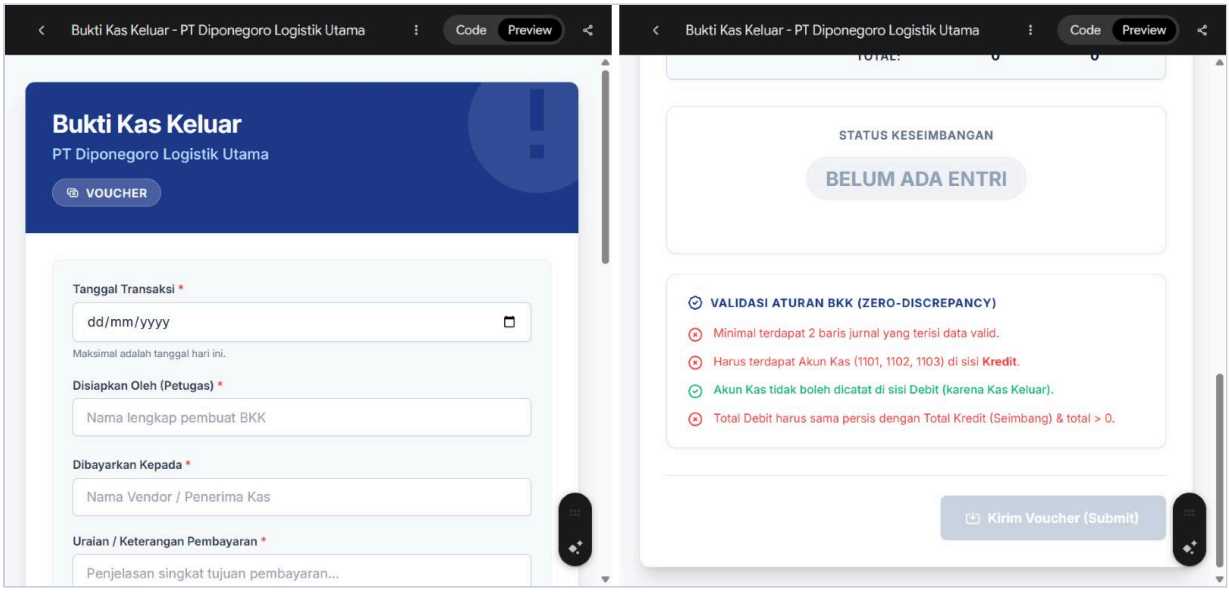
Soal 3 — Purwarupa Web Full-Stack & Orkestrasi AI Tools

3.1a Gemini Canvas: rancangan form dengan live balanced indicator

Prompt instruksi (Gemini Pro + tool Canvas, 13 Oktober 2026):

```
Peran: kamu front-end engineer untuk sistem informasi akuntansi. Buat satu file HTML (Tailwind CSS via CDN
+
JavaScript murni, tanpa framework) berupa formulir Voucher Kas Keluar / Bukti Kas Keluar (BKK) untuk
PT Diponegoro Logistik Utama. Ketentuan: (1) Header voucher: tanggal transaksi (tidak boleh di masa depan),
nama petugas (created_by), dibayarkan kepada, dan uraian. (2) Tabel baris jurnal dinamis: pilih akun, memo,
debit, kredit, tombol tambah/hapus baris; satu baris hanya boleh berisi debit ATAU kredit. Daftar akun
contoh:
1101 Kas Besar, 1102 Kas Kecil, 1103 Bank Mandiri (ketiganya akun kas), 2101 Utang Usaha, 6102 Beban
Listrik
dan Air, 6103 Beban Pengiriman, 6104 Beban Perlengkapan Kantor, 6105 Beban Pemeliharaan Armada. (3) Live
balanced indicator: total debit dan total kredit dihitung otomatis setiap input berubah; tampilkan status
SEIMBANG (hijau) atau BELUM SEIMBANG beserta selisihnya (kuning). Hitung dalam satuan sen (integer) agar
bebas galat floating point. (4) Checklist aturan bisnis yang ikut berubah secara live: minimal 2 baris
lengkap;
ada akun kas di sisi kredit; akun kas tidak boleh didebit (karena ini kas keluar); total debit = total
kredit.
(5) Tombol Submit nonaktif sampai semua aturan terpenuhi (asas zero-discrepancy). Saat submit, tampilkan
payload
JSON {trx_date, prepared_by, counterparty, description, lines:[{account_code, debit, credit, memo}]} yang
nantinya dikirim ke Supabase RPC bernama post_cash_voucher. (6) Format angka Rupiah locale id-ID, tampilan
bersih, dan aksesibel (label pada setiap input).
```

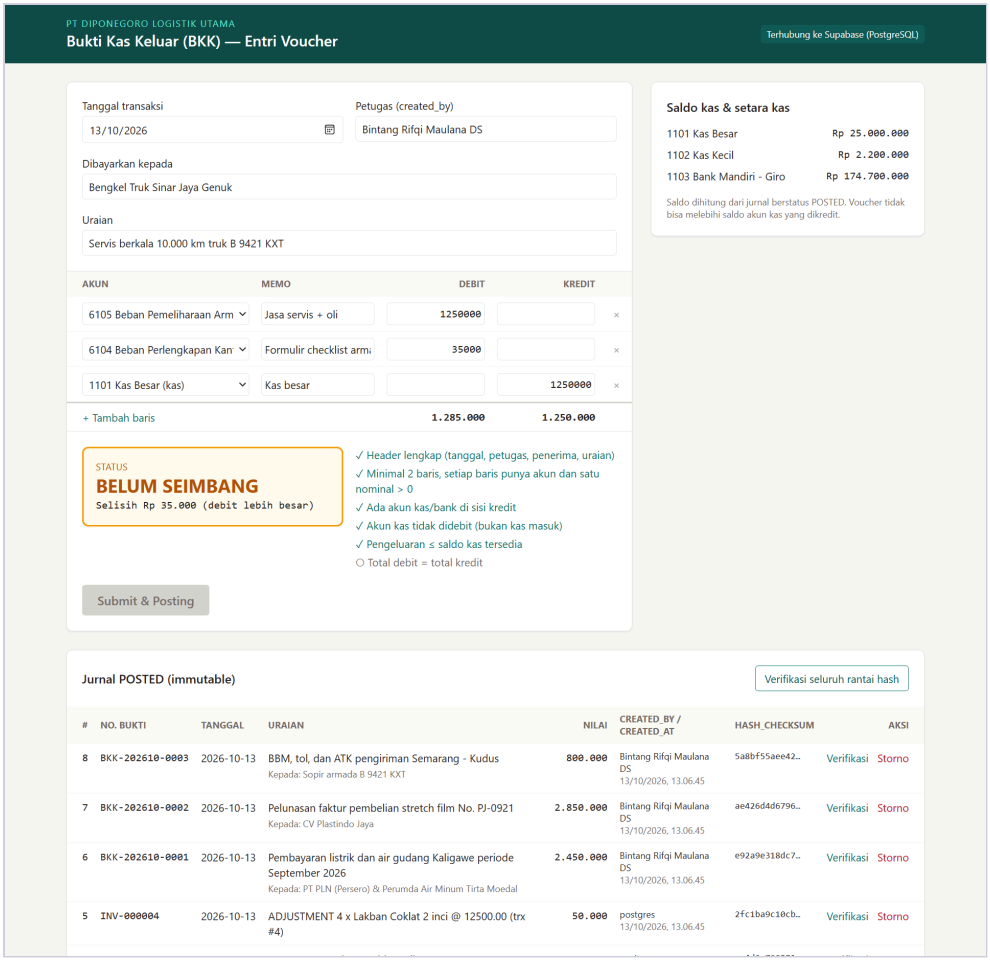
Teknik prompt: *role prompting*, batasan teknologi, aturan bisnis bernomor, **kontrak data** yang disamakan dengan parameter RPC di database, dan syarat non-fungsional (perhitungan dalam sen, locale, aksesibilitas). Respons pertama Gemini menolak ("hanya AI berbasis teks"). Setelah prompt lanjutan *"Ini untuk tugas kuliah Sistem Informasi Akuntansi. Tolong tuliskan kodenya di Canvas…"*, Canvas menghasilkan file HTML ±34 KB.



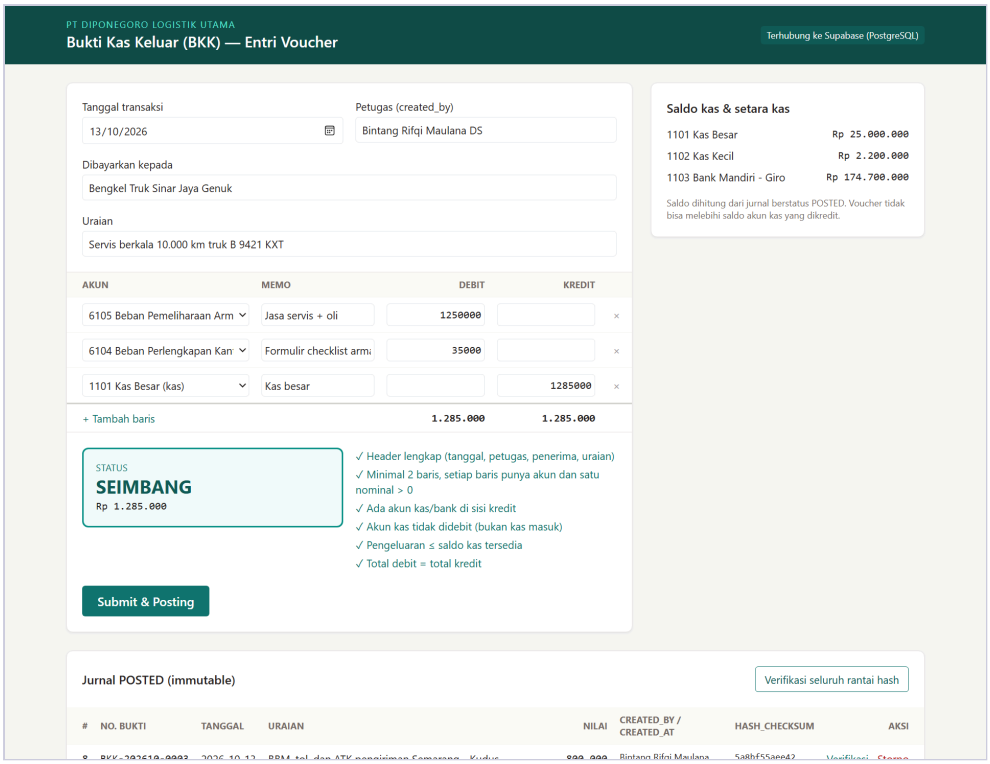
Gambar 1. Hasil Gemini Canvas: header voucher (kiri), status keseimbangan dan checklist aturan BKK, Submit terkunci (kanan).

Review kode Gemini sebelum diadopsi: perhitungan dalam sen dan debit/kredit eksklusif sudah benar. Namun ditemukan celah: nominal pada baris yang belum memilih akun tetap ikut dijumlahkan, sehingga voucher bisa "seimbang" dengan baris tanpa akun. Daftar akun juga di-*hardcode* dan belum ada cek saldo kas. Ketiganya diperbaiki di `index.html` final: setiap baris wajib berakun, COA dibaca dari Supabase, dan

ada aturan "pengeluaran ≤ saldo kas". Satu prinsip dipegang: validasi frontend hanya untuk kenyamanan pengguna, sedangkan **penegakan sebenarnya ada di database**.



Gambar 2. Purwarupa final (GitHub Pages ↔ Supabase): selisih Rp 35.000 → status BELUM SEIMBANG, aturan terakhir belum terpenuhi, tombol Submit nonaktif.

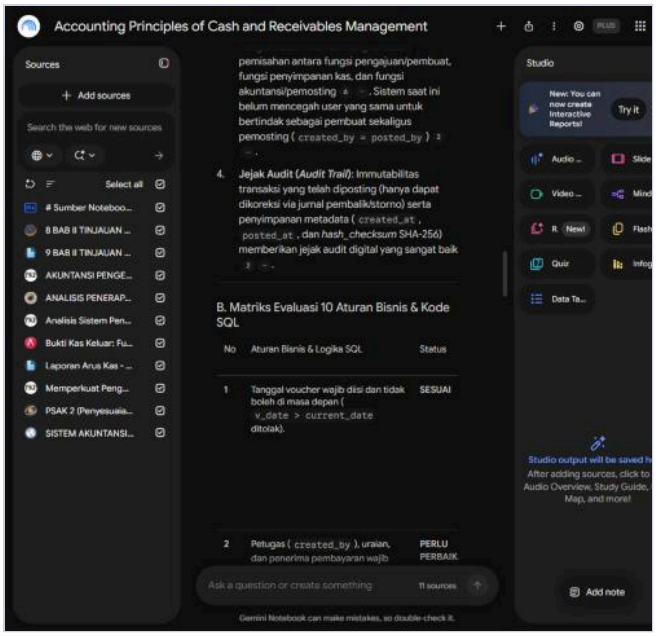


Gambar 3. Setelah kredit Kas Besar = 1.285.000: status SEIMBANG, semua aturan ✓, Submit aktif. Voucher ini kemudian diposting sebagai BKK-202610-0004.

3.1b NotebookLM: verifikasi kepatuhan logika terhadap PSAK (kas dan setara kas)

Sumber notebook (11): 10 sumber web hasil *Fast Research*, antara lain "PSAK 2 (Penyesuaian 2014): Laporan Arus Kas — Accounting BINUS", "Analisis Penerapan Laporan Arus Kas berdasarkan PSAK No. 2", "Bukti Kas Keluar: Fungsi, Tujuan, Jenis — Accurate", dan beberapa jurnal tentang pengendalian internal pengeluaran kas, ditambah 1 dokumen kode berisi aturan bisnis dan SQL `post_cash_voucher` (`ai-logs/notebooklm-sumber-kode.md`).

Prompt: "*Bertindaklah sebagai auditor internal. Berdasarkan sumber PSAK 2 dan literatur pengendalian internal pengeluaran kas di notebook ini, verifikasi apakah 10 aturan bisnis dan kode SQL … sudah patuh terhadap (a) definisi kas dan setara kas …, (b) pengakuan pengeluaran kas dan asas berpasangan debit-kredit, (c) pengendalian internal bukti kas keluar (otorisasi, nomor urut tercetak, pemisahan tugas, jejak audit). Untuk setiap aturan beri status SESUAI atau PERLU PERBAIKAN dengan alasan singkat dan sitasi sumber …*"



Gambar 4. Notebook dengan 11 sumber dan jawaban bersitasi.

Ringkasan verifikasi dari NotebookLM:

- **Kas & setara kas:** 1101 Kas Besar, 1102 Kas Kecil (*cash on hand*), dan 1103 Bank Giro (*demand deposits*) **tepat** sebagai kas menurut PSAK 2. Deposito menjadi setara kas hanya jika jatuh tempo ≤ 3 bulan sejak perolehan. Selebihnya investasi (`is_cash = false`).
- **Pengakuan & double-entry:** kredit kas dengan debit akun lawan dan Σdebit = Σkredit dinilai **sesuai**.
- **Pengendalian internal:** nomor BKK berurutan otomatis memenuhi prinsip *pre-numbered documents*. Imutabilitas, storno, dan hash SHA-256 dinilai "jejak audit digital yang sangat baik". Kekurangannya: belum ada **approval** dan **pemisahan tugas** (created_by = posted_by masih diizinkan).

No	Aturan	Status	Tindak lanjut penyusun
1, 3, 5, 6, 8, 9	Tanggal ≤ hari ini; ≥ 2 baris satu sisi; kredit kas wajib; Σdebit = Σkredit; nomor berurutan; POSTED terkunci + storno	SESUAI	Dipertahankan.
4	Akun kas tidak boleh didebit	PERLU PERBAIKAN	Dipertahankan sebagai desain. Pemindahan antar-kas (mis. pengisian kas kecil dari bank) tidak mengubah total kas dan setara kas, sehingga menurut PSAK 2 bukan arus kas. Transaksi ini dicatat di bukti pemindahbukuan, bukan BKK.
7	Pengeluaran ≤ saldo kas	PERLU PERBAIKAN	Entitas kasus tidak memiliki fasilitas cerukan. Bila ada, tambahkan `overdraft_limit` pada COA.
2, 10	Identitas petugas & jejak audit	PERLU PERBAIKAN	Diakui sebagai batasan purwarupa (demo tanpa login). Produksi: Supabase Auth (`auth.uid()`), status APPROVED, dan aturan `approved_by <> created_by` .

3.2 Mekanisme audit trail dan immutability (Supabase RPC + trigger)

Spesifikasi backend. Klien (anon key) hanya diberi hak `SELECT` lewat RLS. Hak INSERT/UPDATE/DELETE dicabut. Semua penulisan wajib lewat fungsi RPC `security definer` yang memvalidasi ulang:

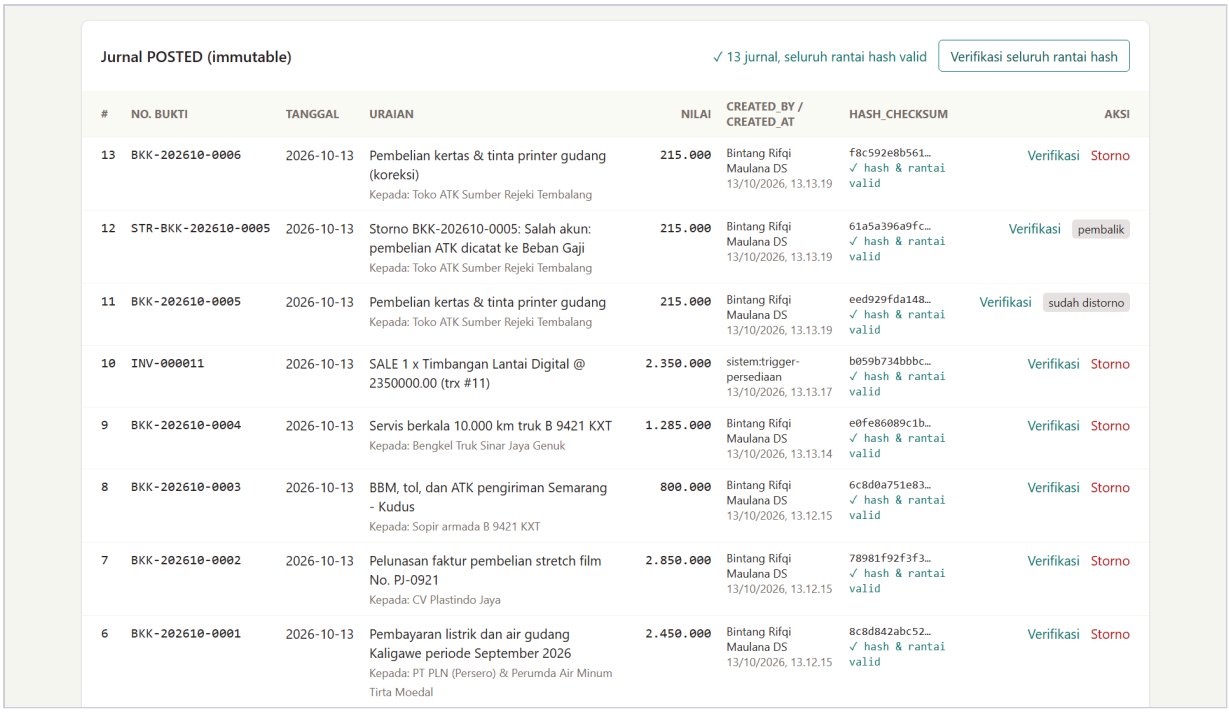
Fungsi / trigger	Perilaku
`post_cash_voucher(jsonb)`	Validasi header, akun terdaftar, maks. 2 desimal, kas tidak didebit, minimal satu kredit kas, Σdebit = Σkredit, dan saldo kas cukup (dengan advisory lock). Nomor BKK-YYYYMM-#### dibuat otomatis, lalu insert DRAFT → lines → UPDATE ke POSTED.

`reverse_journal(id, alasan, petugas)`	Hanya untuk jurnal POSTED yang belum pernah distorno. Membuat jurnal `STR-…` dengan debit/kredit ditukar dan `reversal_of` menunjuk ke jurnal asal. Jurnal asal **tidak diubah**.
`trg_journal_header_guard` (BEFORE INSERT/UPDATE/DELETE)	Insert wajib DRAFT, `created_at := now()` (waktu server). UPDATE/DELETE pada POSTED → `RAISE EXCEPTION … Koreksi wajib melalui jurnal pembalik (storno)`. `created_by/created_at` tidak bisa ditimpa. Saat DRAFT→POSTED: `posted_seq`, `prev_hash`, `posted_at`, dan `hash_checksum` diisi.
`trg_journal_line_guard`	Baris milik jurnal POSTED tidak bisa ditambah, diubah, atau dihapus.
`trg_journal_audit`, `audit_log`	Mencatat CREATE, POST, dan REVERSE (aktor, waktu, no. bukti, hash). `audit_log` sendiri append-only.
`verify_journal(id)`, `verify_ledger()`	Menghitung ulang hash dan memeriksa rantai `prev_hash`.

```
-- hash_checksum = SHA-256( header | baris jurnal | hash jurnal POSTED sebelumnya )
create function journal_hash(h journal_headers) returns char(64) language sql stable as $$
  select encode(sha256(convert_to(concat_ws('|',
      h.header_id, h.trx_date, h.ref_no, h.description, coalesce(h.counterparty,''), h.source,
      coalesce(h.reversal_of::text,''), h.created_by,
      to_char(h.created_at at time zone 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US'), h.posted_by, h.posted_seq,
      (select string_agg(concat_ws(':', a.account_code, l.debit, l.credit), ';' order by l.line_id)
         from journal_lines l join chart_of_accounts a using (account_id) where l.header_id = h.header_id),
      coalesce(h.prev_hash,'GENESIS')), 'UTF8')), 'hex') $$;

-- di trigger guard, transisi DRAFT -> POSTED:
perform pg_advisory_xact_lock(hashtext('gl_posting_chain'));          -- posting serial, rantai tidak
bercabang
select coalesce(max(posted_seq),0) + 1 into new.posted_seq from journal_headers;
select hash_checksum into new.prev_hash from journal_headers where posted_seq = new.posted_seq - 1;
new.posted_at := now();  new.hash_checksum := journal_hash(new);
```

Penjelasan field. `created_by` berisi identitas pembuat. Di purwarupa, nilainya diisi dari nama petugas pada RPC. Pada produksi nilainya diambil dari `auth.uid()`/email JWT sehingga tidak bisa dipalsukan. Setelah insert, trigger mengunci kolom ini. `created_at` selalu diisi `now()` oleh server, sehingga jam klien diabaikan. `hash_checksum` adalah sidik jari SHA-256 dari isi header, seluruh baris, dan hash jurnal sebelumnya. Bila satu angka pada jurnal lama diubah, misalnya oleh DBA yang mematikan trigger, hash jurnal itu tidak cocok lagi dan rantai setelahnya putus. Kondisi ini terdeteksi oleh `verify_ledger()` (uji T16) dan tombol "Verifikasi" di web.



Gambar 5. Daftar jurnal POSTED: "13 jurnal, seluruh rantai hash valid". BKK-202610-0005 "sudah distorno" oleh STR-BKK-202610-0005 dan dikoreksi dengan BKK-202610-0006. Setiap baris menampilkan created_by/created_at dan hash.

5. Pengujian

Fungsi `run_ledger_tests()` menjalankan setiap skenario di dalam sub-transaksi yang selalu di-rollback, sehingga data asli tidak berubah. Hasil di Supabase, 13 Oktober 2026:

Uji	Hasil	Pesan dari database
T01 Jurnal tidak seimbang dikirim langsung (tanpa frontend)	PASS	Jurnal #17 ditolak: tidak seimbang (debit 100000.00, kredit 90000.00, 2 baris)
T02 Baris debit = 0 dan kredit = 0	PASS	violates check constraint "ck_not_both_zero"
T03 Nominal debit negatif	PASS	violates check constraint "ck_debit_non_negative"
T04 Hapus akun yang sudah dipakai (ON DELETE RESTRICT)	PASS	violates foreign key constraint "journal_lines_account_id_fkey"
T05 PURCHASE: stok bertambah, Dr Persediaan / Cr Utang	PASS	stok 296 → 306, Dr Persediaan 125000.00
T06 SALE melebihi stok (anti-negative inventory)	PASS	Stok BRG-01 (Palet Kayu 120x100 cm) tidak cukup: tersedia 28, keluar 29
T07 SALE: stok berkurang, Dr HPP = qty × unit_cost	PASS	stok 28 → 26, Dr HPP 370000.00
T08 UPDATE jurnal POSTED	PASS	Jurnal SA-202610 sudah POSTED: UPDATE ditolak. Koreksi wajib melalui jurnal pembalik (storno).
T09 DELETE jurnal POSTED	PASS	Jurnal SA-202610 sudah POSTED: DELETE ditolak. …
T10 Ubah nominal baris jurnal POSTED	PASS	Baris jurnal milik jurnal POSTED tidak dapat di-UPDATE
T11 Voucher tidak seimbang lewat RPC	PASS	Zero-discrepancy gagal: debit 50000 <> kredit 40000

T12 Voucher tanpa kredit akun kas	PASS	Voucher kas keluar wajib mengkredit minimal satu akun kas/bank
T13 Voucher melebihi saldo kas	PASS	Saldo Kas Kecil tidak mencukupi untuk pengeluaran 99000000
T14 Voucher valid: POSTED, created_by terisi, hash valid	PASS	BKK-202610-0004 hash fba962cf1f52e224…
T15 Storno membalik saldo kas; storno kedua	PASS	saldo Kas Kecil kembali ke 2200000.00; storno ganda ditolak
T16 Manipulasi langsung (trigger dimatikan DBA)	PASS	hash_valid = false setelah deskripsi diubah

Uji dari sisi klien (anon key, sama seperti penyerang lewat API)

Skenario	Hasil
INSERT header langsung ke `journal_headers`	permission denied for table journal_headers
UPDATE / DELETE jurnal SA-202610 lewat REST	permission denied for table journal_headers
RPC voucher debit 500.000 vs kredit 450.000	Zero-discrepancy gagal: debit 500000 <> kredit 450000
2 kasir paralel, SALE BRG-05 (stok 1)	1 berhasil (2.010 ms), 1 ditolak "tersedia 0, keluar 1"; stok akhir 0
Storno BKK-202610-0005 → koreksi BKK-202610-0006	STR-BKK-202610-0005 dibuat; storno kedua ditolak; verify_ledger: 13 jurnal valid

6. Kesimpulan

1. **Integritas saldo dipindahkan ke database.** CHECK per baris, FK RESTRICT, dan constraint trigger yang ditunda sampai COMMIT membuat jurnal tidak seimbang mustahil tersimpan, termasuk yang dikirim langsung lewat API. Ini menjawab akar masalah unbalanced balance sheet pada kasus Soal 1.
2. **Persediaan perpetual real-time.** Trigger memperbarui stok, Persediaan, dan HPP secara atomik. Kunci `FOR UPDATE` mencegah overselling, dan ini terbukti lewat uji paralel.
3. **Audit trail yang dapat diverifikasi.** Jurnal POSTED tidak bisa diubah atau dihapus, koreksi melalui storno, dan hash SHA-256 berantai mendeteksi manipulasi bahkan oleh DBA.
4. **AI sebagai asisten, bukan penentu.** Output Gemini Canvas di-review dan ditemukan celahnya. NotebookLM memberi verifikasi PSAK 2 bersitasi, dan setiap temuannya ditindaklanjuti dengan keputusan yang dijelaskan.